

```
; Assignment: 6
; Aim: Multiplication of two 8-bit hexadecimal numbers
; Name: Neha Biswas
; Date: 27/03/2026
```

```
; ----- MACROS -----
```

```
%macro PRINT 2
    mov rax, 1      ; syscall number for write
    mov rdi, 1      ; file descriptor for stdout
    mov rsi, %1      ; address of buffer to print
    mov rdx, %2      ; number of bytes to print
    syscall          ; invoke kernel
%endmacro
```

```
%macro ACCEPT 2
    mov rax, 0      ; syscall number for read
    mov rdi, 0      ; file descriptor for stdin
    mov rsi, %1      ; buffer to store input
    mov rdx, %2      ; maximum bytes to read
    syscall          ; invoke kernel
%endmacro
```

```
; ----- DATA SECTION -----
```

```
section .data
```

```
msg1 db "Multiplication of two numbers", 10 ; heading message
      db "Name: Neha Biswas",10,"Roll No: 3238",10,"Date: 13-02-2026",10,"Assignment: 3",10
msg1len equ $-msg1 ; length of msg1
```

```
menu db 10, "MENU : ", 10
      db "1. Successive Addition", 10 ; menu option 1
      db "2. Add and Shift", 10 ; menu option 2
      db "3. Exit", 10, "Enter Choice: " ; menu option 3 + prompt
menulen equ $-menu ; length of menu
```

```
msg2 db "Enter number 1: " ; prompt for first number
msg2len equ $-msg2 ; length of msg2
```

```
msg3 db "Enter number 2: " ; prompt for second number
msg3len equ $-msg3 ; length of msg3
```

```
error_msg db "Invalid input!", 10 ; error message
error_msg_len equ $-error_msg ; length of error message
```

```
newline db 10 ; newline character
```

```
; ----- BSS SECTION -----
```

```
section .bss
```

```
num1 resb 1 ; reserve 1 byte for first number
num2 resb 1 ; reserve 1 byte for second number
temp resb 3 ; buffer for input (2 hex digits + newline)
choice resb 2 ; buffer for menu choice
dispbuff resb 4 ; buffer for 4-digit hex result
```

```
; ----- CODE SECTION -----
```

```
section .text
global _start
```

```
_start:
    PRINT msg1, msg1len ; display heading
```

```

main_menu:
    PRINT menu, menulen    ; display menu
    ACCEPT choice, 2       ; read user choice

    cmp byte [choice], '1' ; compare with option '1'
    je call_sa             ; jump if equal

    cmp byte [choice], '2' ; compare with option '2'
    je call_as             ; jump if equal

    cmp byte [choice], '3' ; compare with option '3'
    je exit_prog           ; jump to exit

    jmp main_menu          ; invalid input → repeat menu

```

```

call_sa:
    call successive_addition ; call method 1 procedure
    jmp main_menu           ; return to menu

```

```

call_as:
    call add_and_shift      ; call method 2 procedure
    jmp main_menu           ; return to menu

```

; ----- SUCCESSIVE ADDITION -----

```

successive_addition:

    PRINT msg2, msg2len    ; prompt user for first number
    ACCEPT temp, 3         ; read input
    call ascii_hex8        ; convert ASCII to hex
    mov [num1], bl         ; store result in num1

    PRINT msg3, msg3len    ; prompt for second number
    ACCEPT temp, 3         ; read input
    call ascii_hex8        ; convert ASCII to hex
    mov [num2], bl         ; store result in num2

    xor ax, ax             ; clear AX (result = 0)
    movzx cx, byte [num2]  ; load multiplier into CX

```

```

sa_loop:
    add al, [num1]         ; add num1 to AL
    adc ah, 0              ; add carry to AH
    dec cx                 ; decrement counter
    jnz sa_loop            ; repeat until zero

    mov bx, ax             ; move result to BX
    call display16num       ; display result
    PRINT newline, 1       ; print newline
    ret                    ; return to caller

```

; ----- ADD AND SHIFT -----

```

add_and_shift:

    PRINT msg2, msg2len    ; prompt for first number
    ACCEPT temp, 3         ; read input
    call ascii_hex8        ; convert ASCII to hex
    mov [num1], bl         ; store multiplicand

    PRINT msg3, msg3len    ; prompt for second number
    ACCEPT temp, 3         ; read input
    call ascii_hex8        ; convert ASCII to hex
    mov [num2], bl         ; store multiplier

```

```

    movzx dx, byte [num1] ; load multiplicand into DX
    movzx bx, byte [num2] ; load multiplier into BX
    xor ax, ax             ; clear AX (result)
    mov cl, 8              ; loop for 8 bits

as_loop:
    shr bl, 1              ; shift right, LSB → carry flag
    jnc skip_add           ; if carry = 0, skip addition
    add ax, dx             ; if carry = 1, add multiplicand

skip_add:
    shl dx, 1              ; shift multiplicand left
    dec cl                 ; decrement bit counter
    jnz as_loop            ; repeat loop

    mov bx, ax             ; move result to BX
    call display16num       ; display result
    PRINT newline, 1       ; print newline
    ret                    ; return

```

; ----- ASCII TO HEX -----

```

ascii_hex8:
    mov rdi, temp          ; pointer to input buffer
    xor rbx, rbx           ; clear result register
    mov cl, 2              ; process 2 hex digits

next:
    rol bl, 4              ; shift left 4 bits
    mov al, [rdi]          ; load character

    cmp al, '0'            ; check if below '0'
    jb err                 ; invalid
    cmp al, '9'            ; if 0–9
    jbe digit

    cmp al, 'A'            ; if A–F
    jb err
    cmp al, 'F'
    jbe upper

    cmp al, 'a'
    jb err
    cmp al, 'f'
    jbe lower              ; if a–f

    jmp err                ; otherwise invalid

digit:
    sub al, '0'            ; convert ASCII to number
    jmp store

upper:
    sub al, 55             ; convert A–F to 10–15
    jmp store

lower:
    sub al, 87             ; convert a–f to 10–15

store:
    add bl, al             ; add digit to result
    inc rdi                ; move to next character
    dec cl                 ; decrement counter
    jnz next               ; repeat
    ret                    ; return

```

; ----- DISPLAY HEX -----

```

display16num:
    mov rdi, dispbuff    ; pointer to output buffer
    mov cl, 4            ; 4 hex digits

disp_loop:
    rol bx, 4            ; rotate left 4 bits
    mov al, bl           ; move low byte to AL
    and al, 0Fh          ; mask lower 4 bits

    cmp al, 9
    jbe add30            ; if 0-9
    add al, 7            ; adjust for A-F

add30:
    add al, '0'          ; convert to ASCII
    mov [rdi], al        ; store character
    inc rdi              ; move to next position
    dec cl               ; decrement counter
    jnz disp_loop        ; repeat

    PRINT dispbuff, 4    ; print result
    ret                 ; return

; ----- ERROR -----

err:
    PRINT error_msg, error_msg_len ; print error message
    mov rax, 60          ; exit syscall
    mov rdi, 1           ; exit code 1
    syscall

; ----- EXIT -----

exit_prog:
    mov rax, 60          ; exit syscall
    xor rdi, rdi         ; exit code 0
    syscall

```

Output:

```

neha@LAPTOP-EFHUTG3: /mnt/c/Users/NEHA/OneDrive/Desktop/sppu_notes/com/lab$ nasm -f elf64 3238_asg-6.asm
neha@LAPTOP-EFHUTG3: /mnt/c/Users/NEHA/OneDrive/Desktop/sppu_notes/com/lab$ ld 3238_asg-6.o -o 3238_asg-6
neha@LAPTOP-EFHUTG3: /mnt/c/Users/NEHA/OneDrive/Desktop/sppu_notes/com/lab$ ./3238_asg-6
Multiplication of two numbers
Name: Neha Biswas
Roll No: 3238
Date: 13-02-2026
Assignment: 3

MENU :
1. Successive Addition
2. Add and Shift
3. Exit
Enter Choice: 1
Enter number 1: 7A
Enter number 2: 00
0000

MENU :
1. Successive Addition
2. Add and Shift
3. Exit
Enter Choice: 2
Enter number 1: 08
Enter number 2: 04
0020

MENU :
1. Successive Addition
2. Add and Shift
3. Exit
Enter Choice: 3

```